

Создание первого чатбота

Чат-бот— это программа, которая имитирует реальный разговор с пользователем. Чат-боты позволяют общаться с помощью текстовых или аудио сообщений на сайтах, в мессенджерах, мобильных приложениях

Основные преимущества

- Обеспечивают сервисное обслуживание 24/7
- Помогают охватить больше клиентов
- Экономность
- Легкость в эксплуатации

Как они работают? Большинство людей не будут создавать своих чат-ботов с нуля, так как сегодня существует достаточно большой выбор всевозможных фреймворков и сервисов, которые могут помочь в создании чат-бота. Однако чтобы понять, как они работают нужно погрузиться немного глубже. Бэкенд: Чат-боты могут быть разработаны на любом языке программирования Фронтенд: Это может быть любой мессенджер: от популярных вроде Facebook Messenger, Slack, Telegram до простеньких Realtime Chat With Node.js. Вы не ограничены одной платформой: один и тот же бот может работать, по сути, везде.

Шаг 1. Создаём бота в Telegram

Открываем телеграм и ищем чат @botfather. Сначала пишем /newbot. Затем задаем имя, потом id. После создания бота нам придёт API токен, он нам понадобится в дальнейшем.

 **Nik** 11:07:34 AM
/newbot

 **BotFather** 11:07:35 AM
Alright, a new bot. How are we going to call it? Please choose a name for your bot.

 **Nik** 11:07:51 AM
YO_bot

 **BotFather** 11:07:51 AM
Good. Now let's choose a username for your bot. It must end in `bot`. Like this, for example: TetrisBot or tetris_bot.

 **Nik** edited 11:08:23 AM
YO_pfmexp_bot

 **BotFather** 11:08:23 AM
Выбранный ID не уникален
Sorry, the username must end in 'bot'. E.g. 'Tetris_bot' or 'Tetrisbot'

 **Nik** 11:09:08 AM
PFMEXPBOT

 **BotFather** 11:09:09 AM
Done! Congratulations on your new bot. You will find it at t.me/PFMEXPBOT. You can now add a description, about section and profile picture for your bot, see [/help](#) for a list of commands. By the way, when you've finished creating your cool bot, ping our Bot Support if you want a better username for it. Just make sure the bot is fully operational before you do this.

Use this token to access the HTTP API:
1213457329:AAFFQrCGlLv0b-_U8BWR9w85GLtFomdsak

Keep your token **secure** and **store it safely**, it can be used by anyone to control your bot.

Шаг 2. Пишем основу бота

Устанавливаем библиотеку python-telegram-bot. Ссылка на документацию <https://github.com/python-telegram-bot/python-telegram-bot> (<https://github.com/python-telegram-bot/python-telegram-bot>).

Ввод [13]: !pip install python-telegram-bot==13.8

```
Collecting python-telegram-bot==13.8
  Downloading python_telegram_bot-13.8-py3-none-any.whl (495 kB)
    495.2/495.2 kB 2.9 MB/s eta 0:00:00a 0:00:01
Requirement already satisfied: pytz>=2018.6 in /home/rzaharov@mvs.local/venv375/lib/python3.7/site-packages (from python-telegram-bot==13.8) (2022.1)
Requirement already satisfied: certifi in /home/rzaharov@mvs.local/venv375/lib/python3.7/site-packages (from python-telegram-bot==13.8) (2021.10.8)
Requirement already satisfied: tornado>=6.1 in /home/rzaharov@mvs.local/venv375/lib/python3.7/site-packages (from python-telegram-bot==13.8) (6.1)
Requirement already satisfied: APScheduler==3.6.3 in /home/rzaharov@mvs.local/venv375/lib/python3.7/site-packages (from python-telegram-bot==13.8) (3.6.3)
Requirement already satisfied: cachetools==4.2.2 in /home/rzaharov@mvs.local/venv375/lib/python3.7/site-packages (from python-telegram-bot==13.8) (4.2.2)
Requirement already satisfied: six>=1.4.0 in /home/rzaharov@mvs.local/venv375/lib/python3.7/site-packages (from APScheduler==3.6.3->python-telegram-bot==13.8) (1.15.0)
Requirement already satisfied: setuptools>=0.7 in /home/rzaharov@mvs.local/venv375/lib/python3.7/site-packages (from APScheduler==3.6.3->python-telegram-bot==13.8) (41.2.0)
```

Ввод [2]: !pip install python-telegram-bot --upgrade
!pip install dialogflow
!pip install google-cloud-dialogflow

```
Collecting python-telegram-bot
  Downloading python_telegram_bot-13.11-py3-none-any.whl (497 kB)
    498.0/498.0 kB 3.0 MB/s eta 0:00:00a 0:00:01
Requirement already satisfied: pytz>=2018.6 in /home/rzaharov@mvs.local/venv375/lib/python3.7/site-packages (from python-telegram-bot) (2022.1)
Requirement already satisfied: tornado>=6.1 in /home/rzaharov@mvs.local/venv375/lib/python3.7/site-packages (from python-telegram-bot) (6.1)
Requirement already satisfied: certifi in /home/rzaharov@mvs.local/venv375/lib/python3.7/site-packages (from python-telegram-bot) (2021.10.8)
Collecting cachetools==4.2.2
  Downloading cachetools-4.2.2-py3-none-any.whl (11 kB)
Collecting APScheduler==3.6.3
  Using cached APScheduler-3.6.3-py2.py3-none-any.whl (58 kB)
Collecting tzlocal>=1.2
  Downloading tzlocal-4.2-py3-none-any.whl (19 kB)
Requirement already satisfied: six>=1.4.0 in /home/rzaharov@mvs.local/venv375/lib/python3.7/site-packages (from APScheduler==3.6.3->python-telegram-bot) (1.15.0)
Requirement already satisfied: setuptools>=0.7 in /home/rzaharov@mvs.local/venv375/lib/python3.7/site-packages (from APScheduler==3.6.3->python-telegram-bot) (41.2.0)
```

Явно зададим кодировку нашего кода

telegram.ext подмодуль, предоставляющий простой в использовании интерфейс. Он состоит из нескольких классов, но двумя наиболее важными из них являются telegram.ext.Updater и telegram.ext.Dispatcher. Класс Updater непрерывно получает новые обновления из телеграмма и передает их в Dispatcher класс. Если вы создадите Updater объект, он создаст Dispatcher для вас и свяжет их вместе.

Type *Markdown* and LaTeX: α^2

Затем вы можете зарегистрировать обработчики разных типов в Dispatcher, который будет сортировать обновления, извлеченные в Updater соответствии с зарегистрированными вами обработчиками, и доставлять их в функцию обратного вызова, которую вы определили.

Напишем 2 обработчика команд. Это callback-функции, которые будут вызываться тогда, когда будет получено обновление. Напишем две таких функции для команды /start и для обычного любого текстового сообщения. В качестве аргументов туда передаются два параметра: bot и update. Bot содержит необходимые методы для взаимодействия с API, а update содержит данные о пришедшем сообщении.

Каждый обработчик является экземпляром любого подкласса класса telegram.ext.Handler . Библиотека предоставляет классы-обработчики почти для всех вариантов использования, но если вам нужно что-то очень конкретное, вы также можете создать подкласс Handler самостоятельно.

Теперь осталось лишь присвоить уведомлениям эти обработчики и начать поиск обновлений. Цель состоит в том, чтобы вызывать эту функцию каждый раз, когда бот получает сообщение Telegram, содержащее /start команду. Для этого можно использовать CommandHandler(один из предоставленных подклассов Handler) и зарегистрировать его в диспетчере: Класс Filters содержит ряд функций, которые фильтруют входящие сообщения на наличие текста, изображений, обновлений статуса и т.д. Любое сообщение, которое возвращает True в хотя бы одном из переданных фильтров, MessageHandler примет. Также можно написать свои собственные фильтры.

```
Ввод [1]: from telegram import Update
from telegram.ext import Updater, CommandHandler, MessageHandler, Filters, CallbackContext
```

```
Ввод [2]: # Define a few command handlers. These usually take the two arguments update and
# context. Error handlers also receive the raised TelegramError object in error.
def start(update: Update, context: CallbackContext):
    update.message.reply_text('Hi!')

def echo(update: Update, context: CallbackContext):
    txt = update.message.text

    update.message.reply_text('Ваше сообщение! ' + update.message.text)
```

```
Ввод [3]: updater = Updater("5037721599:AAFxP15Xk0dHR_u2scLj8rA82xR8g1t38qY", use_context=True)
dispatcher = updater.dispatcher

# on different commands - answer in Telegram
dispatcher.add_handler(CommandHandler("start", start))
dispatcher.add_handler(MessageHandler(Filters.text & ~Filters.command, echo))

# Start the Bot
updater.start_polling()
updater.idle()
```

Ввод []:

Ввод []:

Создаём папку Bot, в которой потом создаём файл bot.py. Вы можете создать просто текстовый блокнот и вместо расширения .txt напишите .py Собираем код нашего бота. Открываем консоль и переходим в директорию с файлом, и запускаем python3 bot.py Бот будет работать пока будет открыто окно консоли.

Шаг 3 Dialogflow

<https://dialogflow.cloud.google.com/#/login> (<https://dialogflow.cloud.google.com/#/login>)

Please review your account settings

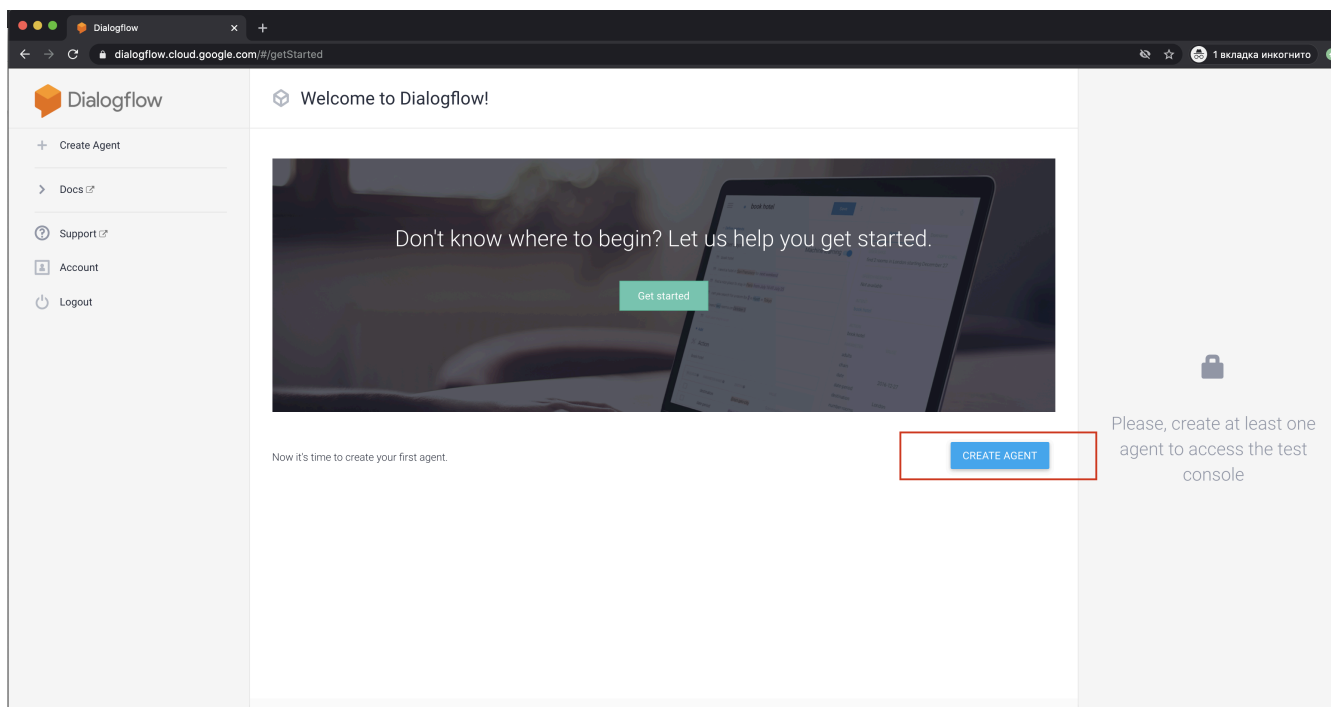
Terms of Service *

✓ Yes, I have read and accept the agreement.

By proceeding and clicking the button below, you agree to adhere to the [Terms of Service](#).

Additionally, you may have access to certain Firebase services. You agree that your use of Firebase services will adhere to the applicable [Firebase Terms of Service](#). If you integrate any apps with Firebase on this project, by default, your Firebase Analytics data will enhance other Firebase features and Google products. You can control how your Firebase Analytics data is shared in your Firebase settings at anytime.

ACCEPT



Ввод []:

DEFAULT LANGUAGE 

Russian — ru

Primary language for your agent. Other languages can be added later.

DEFAULT TIME ZONE

(GMT+3:00) Europe/Moscow

Date and time requests are resolved using this timezone.

GOOGLE PROJECT

New GCP project will be automatically linked to the agent after saving

AGENT TYPE



Set as Mega Agent

Combine multiple Dialogflow agents (i.e. sub agents) into a single agent (i.e. [mega agent](#)).

Панель управления слева. Выбираем Small-talk



Loading agents...



ru



Выбираем Enable и SAVE

Снимок экрана 2020-07-19 в 16.21.00

Small Talk

SAVE

Your agent can learn how to support small talk without any extra development. By default, it will respond with predefined phrases. Use the form below to customize responses to the most popular requests.

User: Как же хочется спать.

Agent: Так закрывай глазки и засыпай. А потом проснёшься и закончишь все дела.

User: Ты очень милый.

Agent: Спасибо! Это взаимно!

☒ Enable



Based on [Actions on Google policy](#), enabling Small Talk in its entirety will cause your action to be rejected. See [Import the prebuilt agent](#) for steps on how to import and select subsets of Small Talk features that comply with Action on Google's policy.

Бот можно протестировать справа

Try it now



Agent

USER SAYS

[COPY CURL](#)

Как ты?



DEFAULT RESPONSE



У меня всё чудесно. Спасибо, что интересуешься.

INTENT

Not available

ACTION

smalltalk.greetings.how_are_you

DIAGNOSTIC INFO

Dialogflow: Python Client

support GA

Python idiomatic client for [Dialogflow](#)

o

[Dialogflow](#) is an enterprise-grade NLU platform that makes it easy for developers to design and integrate conversational user interfaces into mobile apps, web applications, devices, and bots.

- [Dialogflow Python Client API Reference](#)
- [Dialogflow Standard Edition Documentation](#)
- [Dialogflow Enterprise Edition Documentation](#)

Read more about the client libraries for Cloud APIs, including the older Google APIs Client Libraries, in [Client Libraries Explained](#).

Before you begin

1. Select or create a Cloud Platform [project](#).
2. [Enable billing](#) for your project.
3. [Enable the Google Cloud Dialogflow API](#).
4. [Set up authentication](#) with a service account so you can access the API from your local workstation.



Подключение Dialogflow API для приложения в Google Cloud Platform

Google Cloud Platform позволяет управлять приложением и контролировать использование API.

Поскольку у вас нет ни одного проекта, мы создадим новый. Ему будет назначено имя My Project.

Условия использования

☒ Я принимаю [Условия использования Google Cloud Platform](#), а также условия использования [всех соответствующих сервисов и API](#).

Страна проживания

Россия ▼

Я хочу получать по электронной почте новости о продуктах и информацию о специальных предложениях от Google Cloud и партнеров.

☐ Да ☒ Нет

Принять и продолжить

Выбираем свой проект



Активируйте бесплатный пробный период и получите кредит в 300 \$. Вы сможете и



Google Cloud Platform

Выберите проект ▼

Подключение Dialogflow API для приложения в Google Cloud Platform

Google Cloud Platform позволяет управлять приложением и контролировать использование API.

Выберите проект, в котором будет зарегистрировано ваше приложение

Все приложения можно добавить в один проект или создать отдельные проекты для каждого из них.

Создать проект ▼

Все проекты

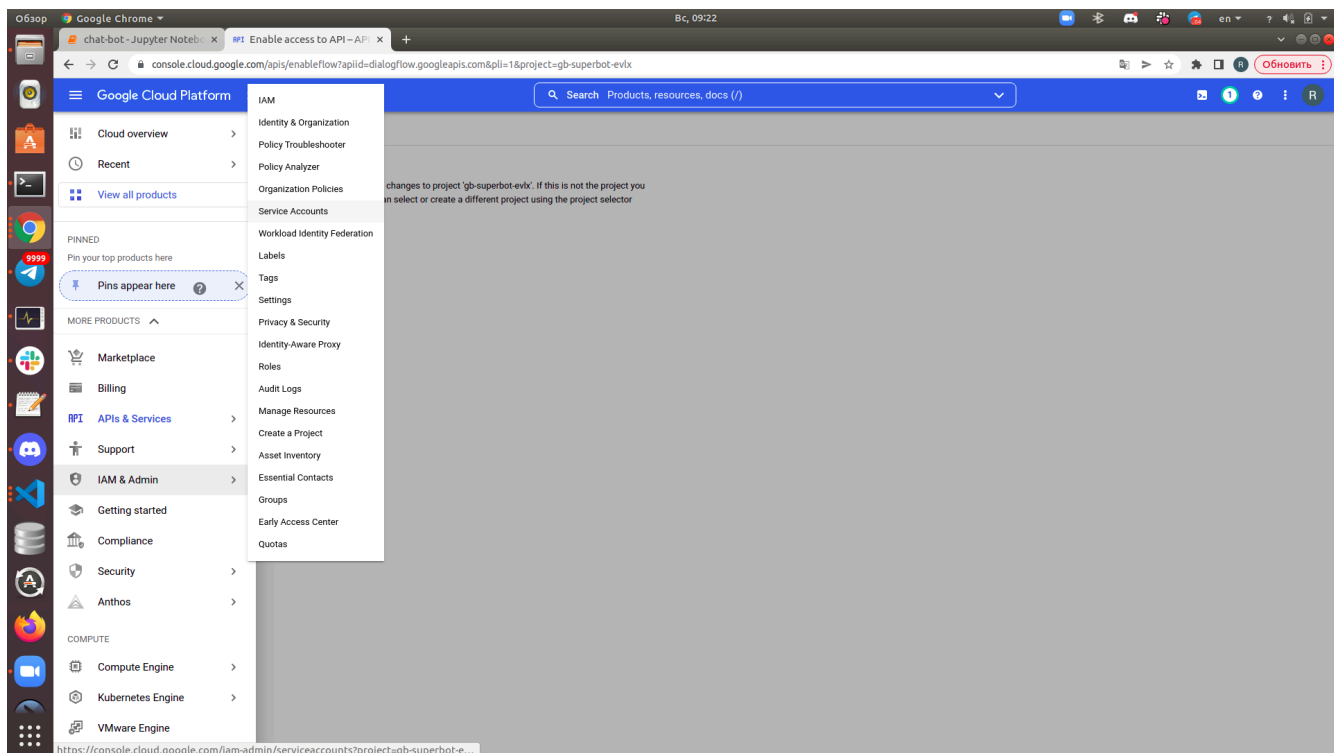
✓ My Project

secure-bonus-283813

Управление проектами

Создать проект

Создаем сервисный аккаунт



Google Chrome | Bc, 09:22

chat-bot - Jupyter Noteb... | Service accounts - IAM &... | console.cloud.google.com/iam-admin/serviceaccounts?project=gb-superbot-evlx

Google Cloud Platform | gb-superbot-evlx | Search | Products, resources, docs (/)

IAM & Admin

- IAM
- Identity & Organization
- Policy Troubleshooter
- Policy Analyzer
- Organization Policies
- Service Accounts
- Workload Identity Federat...
- Labels
- Tags
- Settings
- Privacy & Security
- Identity-Aware Proxy
- Roles
- Audit Logs
- Asset Inventory
- Essential Contacts
- Groups
- Early Access Center
- Manage Resources
- Release Notes

Service accounts

+ CREATE SERVICE ACCOUNT | DELETE | MANAGE ACCESS | REFRESH | HELP ASSISTANT

Service accounts for project "gb-superbot-evlx"

A service account represents a Google Cloud service identity, such as code running on Compute Engine VMs, App Engine apps, or systems running outside Google. [Learn more about service accounts](#)

Organization policies can be used to secure service accounts and block risky service account features, such as automatic IAM Grants, key creation/upload, or the creation of service accounts entirely. [Learn more about service account organization policies](#)

Filter: Enter property name or value

☐	Email	Status	Name	Description	Key ID	Key creation date	OAuth 2.0 Client ID	Actions
☐	gb-superbot@gb-superbot-evlx.iam.gserviceaccount.com	✓	gb-superbot	gb-superbot	b6d7d85a059bc044dcbf87ec1e204b1a3fa2f7a6	Jun 6, 2022	104134717134081590495	

Google Chrome | Bc, 09:23

chat-bot - Jupyter Noteb... | Create service account - | console.cloud.google.com/iam-admin/serviceaccounts/create?project=gb-superbot-evlx

Google Cloud Platform | gb-superbot-evlx | Search | Products, resources, docs (/)

IAM & Admin

- IAM
- Identity & Organization
- Policy Troubleshooter
- Policy Analyzer
- Organization Policies
- Service Accounts
- Workload Identity Federat...
- Labels
- Tags
- Settings
- Privacy & Security
- Identity-Aware Proxy
- Roles
- Audit Logs
- Asset Inventory
- Essential Contacts
- Groups
- Early Access Center
- Manage Resources
- Release Notes

Create service account

HELP ASSISTANT

1 Service account details

Service account name
Display name for this service account

Service account ID * X ↺

Email address: <id>@gb-superbot-evlx.iam.gserviceaccount.com

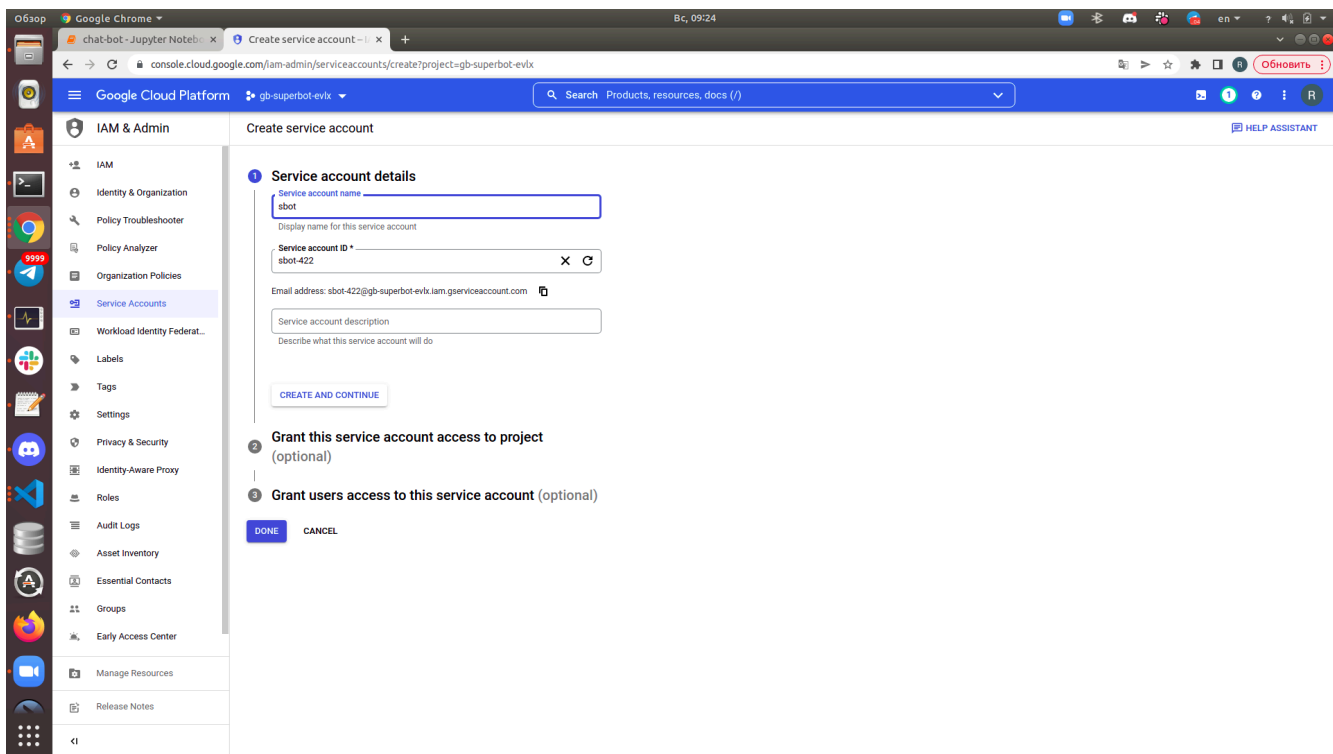
Service account description
Describe what this service account will do

CREATE AND CONTINUE

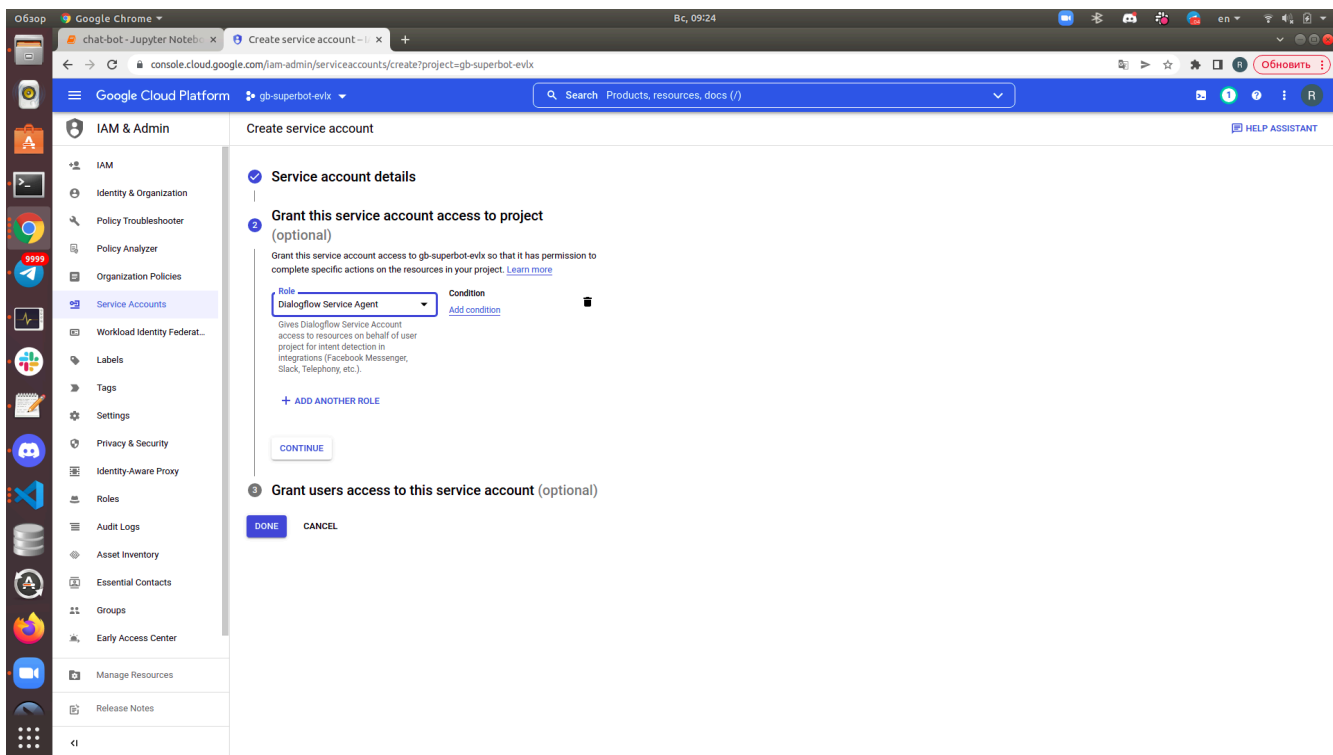
2 Grant this service account access to project (optional)

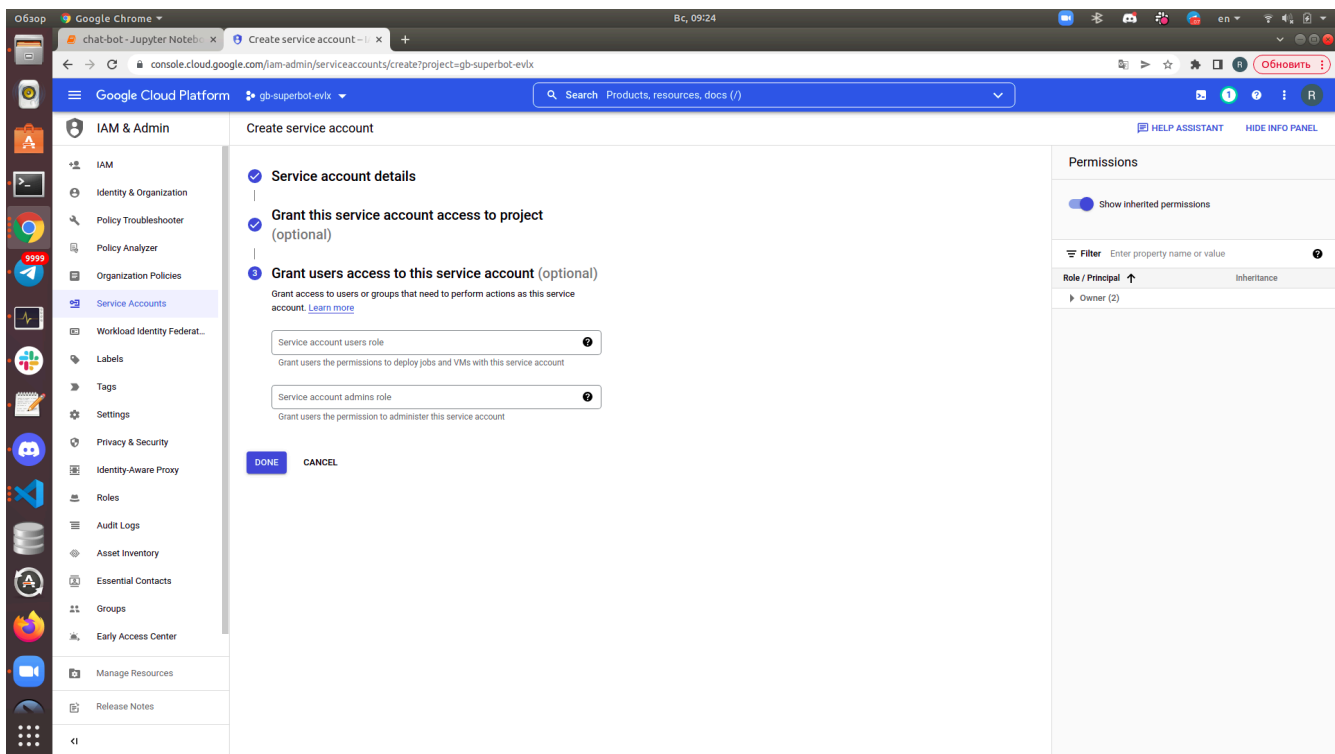
3 Grant users access to this service account (optional)

DONE CANCEL



не забываем добавить роль без неё не будет доступа





Создание ключа

Сервисные аккаунты

Управление сервисными аккаунтами

☐	Эл. почта	Имя ↑	Использование всех сервисов за последние 30 дней ?	
☐	dialogflow-prilqg@bot-qtxvhn.iam.gserviceaccount.com	Dialogflow Integrations	0	

☐	Эл. почта	Состояние	Имя ↑	Описание	Идентификатор ключа	Дата создания ключа	Действия
☐	dialogflow-prilqg@bot-qtxvhn.iam.gserviceaccount.com	✓	Dialogflow Integrations		864ed5ab0c0023ff3b0b922a3850071131af3143	19 июл. 2020 г.	

Изменить

Отключить

Создать ключ

Удалить

вид может быть другим

Google Chrome window showing the Google Cloud Platform IAM & Admin console. The browser tabs include "chat-bot - Jupyter Noteb...", "Service accounts - IAM & Admin", and "console.cloud.google.com/iam-admin/serviceaccounts?project=gb-superbot-evlx". The URL bar shows the console URL. The page title is "Service accounts" with options to "CREATE SERVICE ACCOUNT", "DELETE", "MANAGE ACCESS", and "REFRESH". A "HELP ASSISTANT" button is in the top right.

The left sidebar shows the "IAM & Admin" menu with "Service Accounts" selected. The main content area is titled "Service accounts for project 'gb-superbot-evlx'" and includes a description of service accounts and a link to "Learn more about service account organization policies".

A table lists the service accounts:

Filter	Enter property name or value	Status	Name	Description	Key ID	Key creation date	OAuth 2.0 Client ID	Actions
☐	Email							
☐	gb-superbot@gb-superbot-evlx.iam.gserviceaccount.com	✓	gb-superbot	gb-superbot	b6d7d85a059bc044dc8f87ec1e204b1a3fa27fa6	Jun 6, 2022	104134717134081590495	
☐	sbot-422@gb-superbot-evlx.iam.gserviceaccount.com	✓	sbot		No keys		103151568947892269394	

A context menu is open over the "sbot" service account, showing options: "Manage details", "Manage permissions", "Manage keys", "View metrics", "View logs", "Disable", and "Delete".

The URL bar shows the full path to the keys page: `https://console.cloud.google.com/iam-admin/serviceaccounts/details/103151568947892269394/keys?project=gb-superbot-evlx`

Google Chrome window showing the Google Cloud Platform IAM & Admin console, specifically the "Keys" page for the "sbot" service account. The browser tabs include "chat-bot - Jupyter Noteb...", "sbot - IAM & Admin - gb-", and "console.cloud.google.com/iam-admin/serviceaccounts/details/103151568947892269394/keys?project=gb-superbot-evlx". The URL bar shows the console URL.

The page title is "sbot" with tabs for "DETAILS", "PERMISSIONS", "KEYS", "METRICS", and "LOGS". The "KEYS" tab is selected.

The main content area is titled "Keys" and includes a warning: "Service account keys could pose a security risk if compromised. We recommend you avoid downloading service account keys and instead use the Workload Identity Federation. You can learn more about the best way to authenticate service accounts on Google Cloud here."

Below the warning, there is a section for adding new keys:

Add a new key pair or upload a public key certificate from an existing key pair.

Block service account key creation using organization policies. [Learn more about setting organization policies for service accounts](#)

Buttons: "ADD KEY", "Create new key", "Upload existing key".

Fields: "Key creation date", "Key expiration date".

Создание закрытого ключа для сервисного аккаунта Dialogflow Integrations

На ваш компьютер будет скачан файл с закрытым ключом. Сохраните его в надежном месте. В случае утери его нельзя будет восстановить.

Тип ключа

☒ JSON

Рекомендуемый формат.

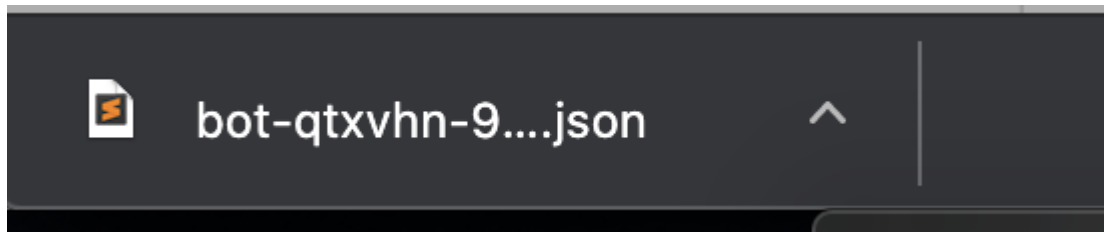
☐ P12

Для совместимости с кодом, который работает с ключами формата P12.

ОТМЕНА

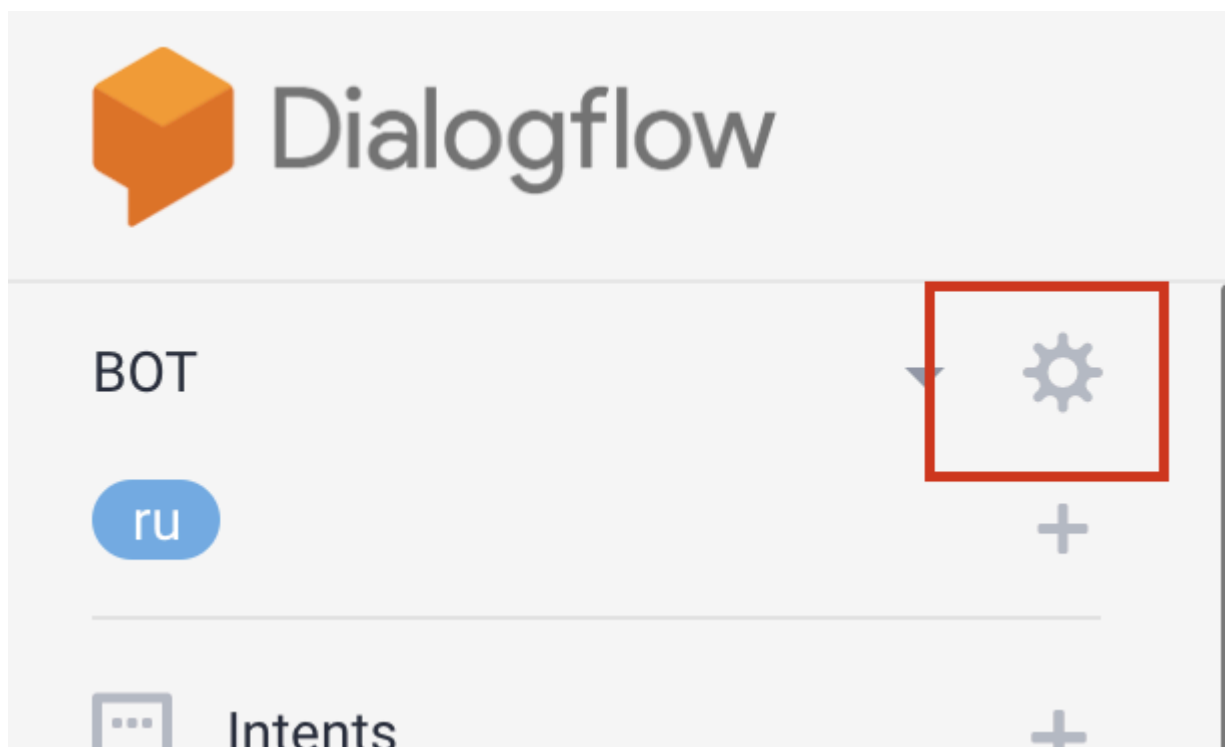
СОЗДАТЬ

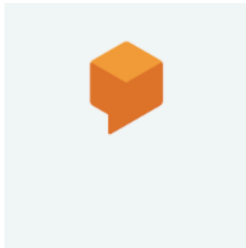
JSON должен скачаться. Выберите директорию с ботом



Шаг 4 Настройки соединения

Перейдем в настройки в DialogFlow и скопируем PROJECT ID





DESCRIPTION

Describe your agent

DEFAULT TIME ZONE

(GMT+3:00) Europe/Moscow

Date and time requests are resolved using this timezone.

GOOGLE PROJECT

Project ID	bot-qtxvhn
Service Account	dialogflow-prilqg@bot-qtxvhn.iam.gserviceaccount.com

Ввод [66]: `!pip install google-cloud-dialogflow`

Collecting google-cloud-dialogflow

Using cached google_cloud_dialogflow-2.14.0-py2.py3-none-any.whl (2.3 MB)

Requirement already satisfied: google-api-core[grpc]!=2.0.*,!=2.1.*,!=2.2.*,!=2.3.0,<3.0.0dev,>=1.31.5 in /home/rzaharov@mvs.local/venv375/lib/python3.7/site-packages (from google-cloud-dialogflow) (2.8.1)

Requirement already satisfied: proto-plus>=1.15.0 in /home/rzaharov@mvs.local/venv375/lib/python3.7/site-packages (from google-cloud-dialogflow) (1.20.4)

Requirement already satisfied: protobuf<4.0.0dev,>=3.15.0 in /home/rzaharov@mvs.local/venv375/lib/python3.7/site-packages (from google-api-core[grpc]!=2.0.*,!=2.1.*,!=2.2.*,!=2.3.0,<3.0.0dev,>=1.31.5->google-cloud-dialogflow) (3.20.1)

Requirement already satisfied: google-auth<3.0dev,>=1.25.0 in /home/rzaharov@mvs.local/venv375/lib/python3.7/site-packages (from google-api-core[grpc]!=2.0.*,!=2.1.*,!=2.2.*,!=2.3.0,<3.0.0dev,>=1.31.5->google-cloud-dialogflow) (2.6.6)

Requirement already satisfied: requests<3.0.0dev,>=2.18.0 in /home/rzaharov@mvs.local/venv375/lib/python3.7/site-packages (from google-api-core[grpc]!=2.0.*,!=2.1.*,!=2.2.*,!=2.3.0,<3.0.0dev,>=1.31.5->google-cloud-dialogflow) (2.27.1)

Requirement already satisfied: googleapis-common-protos<2.0dev,>=1.56.2 in /home/rzaharov@mvs.local/venv375/lib/python3.7/site-packages (from google-api-core[grpc]!=2.0.*,!=2.1.*,!=2.2.*,!=2.3.0,<3.0.0dev,>=1.31.5->google-cloud-dialogflow) (1.56.2)

Ввод [63]: `!pip install grpcio-status`

Collecting grpcio-status

Using cached grpcio_status-1.46.3-py3-none-any.whl (10.0 kB)

Requirement already satisfied: protobuf>=3.12.0 in /home/rzaharov@mvs.local/venv375/lib/python3.7/site-packages (from grpcio-status) (3.20.1)

Requirement already satisfied: googleapis-common-protos>=1.5.5 in /home/rzaharov@mvs.local/venv375/lib/python3.7/site-packages (from grpcio-status) (1.56.2)

Requirement already satisfied: grpcio>=1.46.3 in /home/rzaharov@mvs.local/venv375/lib/python3.7/site-packages (from grpcio-status) (1.46.3)

Requirement already satisfied: six>=1.5.2 in /home/rzaharov@mvs.local/venv375/lib/python3.7/site-packages (from grpcio>=1.46.3->grpcio-status) (1.15.0)

Installing collected packages: grpcio-status

Successfully installed grpcio-status-1.46.3

Ввод [58]: `!pip install google-api-core`

```
Collecting google-api-core
  Using cached google_api_core-2.8.1-py3-none-any.whl (114 kB)
Requirement already satisfied: protobuf<4.0.0dev,>=3.15.0 in /home/rzaharov@mvs.local/venv375/lib/python3.7/site-packages (from google-api-core) (3.20.1)
Requirement already satisfied: googleapis-common-protos<2.0dev,>=1.56.2 in /home/rzaharov@mvs.local/venv375/lib/python3.7/site-packages (from google-api-core) (1.56.2)
Requirement already satisfied: requests<3.0.0dev,>=2.18.0 in /home/rzaharov@mvs.local/venv375/lib/python3.7/site-packages (from google-api-core) (2.27.1)
Requirement already satisfied: google-auth<3.0dev,>=1.25.0 in /home/rzaharov@mvs.local/venv375/lib/python3.7/site-packages (from google-api-core) (2.6.6)
Requirement already satisfied: pyasn1-modules<=0.2.1 in /home/rzaharov@mvs.local/venv375/lib/python3.7/site-packages (from google-auth<3.0dev,>=1.25.0->google-api-core) (0.2.8)
Requirement already satisfied: rsa<5,>=3.1.4 in /home/rzaharov@mvs.local/venv375/lib/python3.7/site-packages (from google-auth<3.0dev,>=1.25.0->google-api-core) (4.8)
Requirement already satisfied: six<=1.9.0 in /home/rzaharov@mvs.local/venv375/lib/python3.7/site-packages (from google-auth<3.0dev,>=1.25.0->google-api-core) (1.15.0)
Requirement already satisfied: cachetools<6.0,>=2.0.0 in /home/rzaharov@mvs.local/venv375/lib/python3.7/site-packages (from google-auth<3.0dev,>=1.25.0->google-api-core) (4.2.4)
```

Ввод [56]: `!pip install google-cloud-vision`

```
Collecting google-cloud-vision
  Downloading google_cloud_vision-2.7.2-py2.py3-none-any.whl (383 kB)
  383.8/383.8 kB 2.9 MB/s eta 0:00:00a 0:00:01
Requirement already satisfied: proto-plus<=1.15.0 in /home/rzaharov@mvs.local/venv375/lib/python3.7/site-packages (from google-cloud-vision) (1.20.4)
Requirement already satisfied: google-api-core[grpc]!=2.0.*,!=2.1.*,!=2.2.*,!=2.3.0,<3.0.0dev,>=1.31.5 in /home/rzaharov@mvs.local/venv375/lib/python3.7/site-packages (from google-cloud-vision) (2.8.1)
Requirement already satisfied: google-auth<3.0dev,>=1.25.0 in /home/rzaharov@mvs.local/venv375/lib/python3.7/site-packages (from google-api-core[grpc]!=2.0.*,!=2.1.*,!=2.2.*,!=2.3.0,<3.0.0dev,>=1.31.5->google-cloud-vision) (2.6.6)
Requirement already satisfied: protobuf<4.0.0dev,>=3.15.0 in /home/rzaharov@mvs.local/venv375/lib/python3.7/site-packages (from google-api-core[grpc]!=2.0.*,!=2.1.*,!=2.2.*,!=2.3.0,<3.0.0dev,>=1.31.5->google-cloud-vision) (3.20.1)
Requirement already satisfied: googleapis-common-protos<2.0dev,>=1.56.2 in /home/rzaharov@mvs.local/venv375/lib/python3.7/site-packages (from google-api-core[grpc]!=2.0.*,!=2.1.*,!=2.2.*,!=2.3.0,<3.0.0dev,>=1.31.5->google-cloud-vision) (1.56.2)
Requirement already satisfied: requests<3.0.0dev,>=2.18.0 in /home/rzaharov@mvs.local/venv375/lib/python3.7/site-packages (from google-api-core[grpc]!=2.0.*,!=2.1.*,!=2.2.*,!=2.3.0,<3.0.0dev,>=1.31.5->google-cloud-vision) (2.27.1)
```

Ввод [6]: `import os`
`import logging`
`from telegram import Update`
`from telegram.ext import Updater, CommandHandler, MessageHandler, Filters, CallbackContext`

`#import dialogflow`

Ввод [5]: `!ls *.json -lah`

```
-rw-r--r-- 1 rzaharov domain users 2.3K Jun 22 21:38 botnlp-super-mxxt-ff54143f23e3.json
-rw-r--r-- 1 rzaharov domain users 2.3K Jun  4 09:38 gb-superbot-evlx-8ddaaab87296.json
-rw-r--r-- 1 rzaharov domain users 2.3K Jun 12 09:25 gb-superbot-evlx-b118a1fd4b25.json
-rw-r--r-- 1 rzaharov domain users 2.3K Jun  6 03:00 gb-superbot-evlx-b6d7d85a059b.json
```

```
Ввод [7]: updater = Updater("5037721599:AAFxP15Xk0dHR_u2scLj8rA82xR8g1t38qY", use_context=True)
dispatcher = updater.dispatcher
os.environ["GOOGLE_APPLICATION_CREDENTIALS"] = 'botnlp-super-mxxt-ff54143f23e3.json'

DIALOGFLOW_PROJECT_ID = 'botnlp-super-mxxt' #PROJECT ID из DialogFlow
DIALOGFLOW_LANGUAGE_CODE = 'ru' # язык
SESSION_ID = 'GB_NLP12_21_bot' # ID бота из телеграма
```

Переписываем функцию textMessage

```
Ввод [8]: from google.cloud import dialogflow
```

```
Ввод [9]: def detect_intent_texts(project_id, session_id, text, language_code):
    session_client = dialogflow.SessionsClient()
    session = session_client.session_path(project_id, session_id)

    text_input = dialogflow.TextInput(text=text, language_code=language_code)
    query_input = dialogflow.QueryInput(text=text_input)

    response = session_client.detect_intent(
        request={"session": session, "query_input": query_input}
    )

    return response.query_result.fulfillment_text
```

```
Ввод [10]: def startCommand(update: Update, context: CallbackContext):
    update.message.reply_text('Добрый день!')

def textMessage(update: Update, context: CallbackContext):
    text_user = update.message.text

    text_bot = detect_intent_texts(DIALOGFLOW_PROJECT_ID, SESSION_ID, text_user, DIALOGFLOW_LANGUAGE_CODE)
    if text_bot:
        update.message.reply_text(text_bot)
    else:
        update.message.reply_text('Что?')
```

```
Ввод [11]: # on different commands - answer in Telegram
dispatcher.add_handler(CommandHandler("start", startCommand))
dispatcher.add_handler(MessageHandler(Filters.text & ~Filters.command, textMessage))
```

```
Ввод [ ]: # Start the Bot
updater.start_polling()
updater.idle()
```

```
Ввод [ ]:
```

```
Ввод [ ]:
```

```
Ввод [ ]:
```

